Numerical Methods Lab

Lab 11 – 18 | C Programs

C Source Code

Lab-11: Newton's Forward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's forward difference. Data: x=1,1.2,1.4,1.6 y=0,0.128,0.544,1.296 at x=1.1

C Program:

```
#include <stdio.h>
#define MAX 20

int main() {
int n, i, j;
float x[MAX], y[MAX][MAX], h, xp, u, dy;

printf("Enter number of data points: ");
scanf("%d", &n);
printf("Enter x and y values (x y):\n");
for (i = 0; i < n; i++) {
printf("x[%d] y[%d]: ", i, i);
scanf("%f %f", &x[i], &y[i][0]);
}
printf("Enter the value of x at which derivative is required: ");
scanf("%f", &xp);

for (j = 1; j < n; j++)
for (i = 0; i < n - j; i++)
y[i][j] = y[i+1][j-1] - y[i][j-1];

h = x[1] - x[0];
u = (xp - x[0]) / h;
dy = (y[0][1] + ((2*u-1)/2.0)*y[0][2] + ((3*u*u-6*u+2)/6.0)*y[0][3]) / h;

printf("\nFirst derivative at x = %.1f is %.4f\n", xp, dy);
return 0;
}
```

Lab-12: Newton's Backward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's backward difference. Data: x=1,1.2,1.4,1.6 y=0,0.128,0.544,1.296 at x=1.1

C Program:

```
#include <stdio.h>
#define MAX 20

int main() {
int n, i, j;
float x[MAX], diff[MAX][MAX], h, xp, u, dy;

printf("Enter number of data points: ");
scanf("%d", &n);
printf("Enter x and y values (x y):\n");
for (i = 0; i < n; i++) {
printf("x[%d] y[%d]: ", i, i);
scanf("%f %f", &x[i], &diff[i][0]);
}
printf("Enter the value of x at which derivative is required: ");
scanf("%f", &xp);

for (j = 1; j < n; j++)
for (i = n-1; i >= j; i--)
diff[i][j] = diff[i][j-1] - diff[i-1][j-1];

h = x[1] - x[0];
u = (xp - x[n-1]) / h;
dy = (diff[n-1][1] + ((2*u+1)/2.0)*diff[n-1][2]
+ ((3*u*u+6*u+2)/6.0)*diff[n-1][3]) / h;

printf("First derivative at x = %.1f is %.4f\n", xp, dy);
return 0;
}
```

Lab-13: Simpson's 1/3 Rule

Question: Write a Python and C program for Composite Simpson's 1/3 Rule. f(x) = x^2 + 2x + 1, x0=0, xn=4, segments=4

C Program:

```
#include <stdio.h>

float f(float x) { return x*x + 2*x + 1; }

int main() {
float x0, xn, h, term;
int k, i;

printf("Function: f(x) = x^2 + 2x + 1\n");
printf("Enter lower limit (x0): "); scanf("%f", &x0);
printf("Enter upper limit (xn): "); scanf("%f", &xn);
printf("Enter number of segments: "); scanf("%d", &k);

if (k % 2 != 0) { printf("Warning: k must be even.\n"); return 1; }

h = (xn - x0) / k;
term = f(x0) + f(xn);
for (i = 1; i < k; i += 2) term += 4 * f(x0 + i*h);
for (i = 2; i < k-1; i += 2) term += 2 * f(x0 + i*h);

printf("Approximate Integral = %.6f\n", (h/3)*term);
return 0;
}
```

Lab-14: Simpson's 3/8 Rule

Question: Write a Python and C program for Simpson's 3/8 Rule. f(x) = x^2 + 2x + 1, x0=0, xn=3, segments=3

C Program:

```
#include <stdio.h>

float f(float x) { return x*x + 2*x + 1; }

int main() {
float x0, xn, h, term;
int k, i;

printf("Function: f(x) = x^2 + 2x + 1\n");
printf("Enter lower limit (x0): "); scanf("%f", &x0);
printf("Enter upper limit (xn): "); scanf("%f", &xn);
printf("Enter number of segments: "); scanf("%d", &k);

h = (xn - x0) / k;
term = f(x0) + f(xn);
for (i = 1; i < k; i++) {
if (i % 3 != 0) term += 3 * f(x0 + i*h);
else term += 2 * f(x0 + i*h);
}
printf("Approximate integral using Simpson's 3/8 Rule: %.6f\n", (3.0/8)*h*term);
return 0;
}
```

Lab-15: Romberg Integration

Question: Write a Python and C program for Romberg Integration. Integrand: sin(x)/x, x0=0, xn=1, p=4 refinements, q=3 extrapolations

C Program:

```
#include <stdio.h>
#include <math.h>
#define PMAX 10

double f(double x) { return (x == 0) ? 1.0 : sin(x)/x; }

int main() {
double a, b, T[PMAX+1][PMAX+1] = {0};
int p, q, i, k;

printf("Enter lower limit (x0): "); scanf("%lf", &a);
printf("Enter upper limit (xn): "); scanf("%lf", &b);
printf("Enter number of trapezoidal refinements (p): "); scanf("%d", &p);
printf("Enter number of extrapolations (q): "); scanf("%d", &q);

double h = b - a;
T[0][0] = (h/2)*(f(a)+f(b));
for (i = 1; i <= p; i++) {
double hi = h / (1 << i), s = 0;
for (k = 1; k <= (1 << (i-1)); k++)
s += f(a + (2*k-1)*hi);
T[i][0] = T[i-1][0]/2 + hi*s;
}
for (int c = 1; c <= p; c++)
for (k = 1; k <= c && k <= q; k++) {
double f4k = pow(4, k);
T[c][k] = (f4k*T[c][k-1] - T[c-1][k-1]) / (f4k - 1);
}

printf("Target Romberg estimate T[%d][%d]\n", p, q);
printf("Approximate integral of sin(x)/x from %.1f to %.1f: %.10f\n", a, b, T[p][q]);
printf("\nRomberg Table (T[i][j]):\n");
for (i = 0; i <= p; i++) {
printf("T[%d]: ", i);
for (k = 0; k <= q; k++) printf("%.10f ", T[i][k]);
printf("\n");
}
return 0;
}
```

Lab-16: Gaussian Elimination

Question: Write a Python and C program for Gaussian Elimination. System: x+y+z=6, 2x+3y+z=14, x+2y+3z=14

C Program:

```
#include <stdio.h>
#define N 10

int main() {
int n, i, j, k;
float A[N][N], B[N], x[N], ratio, sum;

printf("Enter order of square matrix (n): "); scanf("%d", &n);
printf("Enter coefficients of the matrix:\n");
for (i = 0; i < n; i++)
for (j = 0; j < n; j++) {
printf("A[%d][%d]: ", i+1, j+1); scanf("%f", &A[i][j]);
}
printf("\nEnter constants of vector b:\n");
for (i = 0; i < n; i++) { printf("b[%d]: ", i+1); scanf("%f", &B[i]); }

for (i = 0; i < n; i++)
for (j = i+1; j < n; j++) {
ratio = A[j][i] / A[i][i];
for (k = i; k < n; k++) A[j][k] -= ratio * A[i][k];
B[j] -= ratio * B[i];
}
for (i = n-1; i >= 0; i--) {
sum = 0;
for (j = i+1; j < n; j++) sum += A[i][j]*x[j];
x[i] = (B[i] - sum) / A[i][i];
}
printf("\nSolution of the system:\n");
printf("x = %.2f\ny = %.2f\nz = %.2f\n", x[0], x[1], x[2]);
return 0;
}
```

Lab-17: Gaussian Elimination with Partial Pivoting

Question: Write a Python and C program for Gaussian Elimination with Partial Pivoting. System: x+y+z=6, 2x+3y+z=14, x+2y+3z=14

C Program:

```
#include <stdio.h>
#include <math.h>
#define N 10

int main() {
int n, i, j, k, max_row;
float A[N][N], B[N], x[N], tmp, ratio, sum;

printf("Enter order of square matrix (n): "); scanf("%d", &n);
printf("Enter coefficients of 3x3 matrix:\n");
for (i = 0; i < n; i++)
for (j = 0; j < n; j++) {
printf("A[%d][%d] = ", i+1, j+1); scanf("%f", &A[i][j]);
}
printf("\nEnter constants vector:\n");
for (i = 0; i < n; i++) { printf("b[%d] = ", i+1); scanf("%f", &B[i]); }

for (i = 0; i < n; i++) {
max_row = i;
for (k = i+1; k < n; k++)
if (fabs(A[k][i]) > fabs(A[max_row][i])) max_row = k;
for (j = 0; j < n; j++) { tmp=A[i][j]; A[i][j]=A[max_row][j]; A[max_row][j]=tmp; }
tmp=B[i]; B[i]=B[max_row]; B[max_row]=tmp;

for (j = i+1; j < n; j++) {
ratio = A[j][i]/A[i][i];
for (k = i; k < n; k++) A[j][k] -= ratio*A[i][k];
B[j] -= ratio*B[i];
}
}
for (i = n-1; i >= 0; i--) {
sum = 0;
for (j = i+1; j < n; j++) sum += A[i][j]*x[j];
x[i] = (B[i]-sum)/A[i][i];
}
printf("\nSolution of the system:\n");
printf("x = %d\ny = %d\nz = %d\n", (int)roundf(x[0]),(int)roundf(x[1]),(int)roundf(x[2]));
return 0;
}
```

Lab-18: Gauss-Jordan Elimination

Question: Write a Python and C program for Gauss-Jordan Elimination. System: x+y+z=6, 2x+3y+z=14, x+2y+3z=14

C Program:

```
#include <stdio.h>
#include <math.h>
#define N 10

int main() {
int n, i, j, k, max_row;
float A[N][N], B[N], tmp, ratio, pivot;

printf("Enter order of square matrix (n): "); scanf("%d", &n);
printf("Enter coefficients of 3x3 matrix:\n");
for (i = 0; i < n; i++)
for (j = 0; j < n; j++) {
printf("A[%d][%d] = ", i+1, j+1); scanf("%f", &A[i][j]);
}
printf("\nEnter constants vector:\n");
for (i = 0; i < n; i++) { printf("b[%d] = ", i+1); scanf("%f", &B[i]); }

for (i = 0; i < n; i++) {
max_row = i;
for (k = i+1; k < n; k++)
if (fabs(A[k][i]) > fabs(A[max_row][i])) max_row = k;
for (j = 0; j < n; j++) { tmp=A[i][j]; A[i][j]=A[max_row][j]; A[max_row][j]=tmp; }
tmp=B[i]; B[i]=B[max_row]; B[max_row]=tmp;

pivot = A[i][i];
for (k = i; k < n; k++) A[i][k] /= pivot;
B[i] /= pivot;

for (j = 0; j < n; j++) {
if (i != j) {
ratio = A[j][i];
for (k = i; k < n; k++) A[j][k] -= ratio*A[i][k];
B[j] -= ratio*B[i];
}
}
}
printf("\nGauss-Jordan Solution:\n");
printf("x = %d\ny = %d\nz = %d\n", (int)roundf(B[0]),(int)roundf(B[1]),(int)roundf(B[2]));
return 0;
}
```